

PROGRAMMING CONTEST #1

CSE110

Table of Contents

Rules and Regulations	1
App #1: Rock-Paper-Scissors	2
App #2: Tic-Tac-Toe	3
App #3: Aladeen Date Validator	4
App #4: Password Validator	5
App #5: Triangle/Quadrilateral Validator using Sides	6
App #6: Triangle/ Quadrilateral Detector using Angles	7
App #7: Ludo	8
App #8: Super Calculator	9
App #9: Penalty Shootout	11
App #10: Pokémon Battle	12

Rules and Regulations

Team Building Rules

1. There will be eight teams. Eight horsemen/horsewomen will be selected by the instructor. The selection will be based on the class performance of the students. They will form teams according to their preference. Every student must participate.
2. Each team will have 5 members. Each team must have at least one male and one female student.
3. Each team must have at least one person with knowledge on the following:
 - a. Rock-Paper-Scissors
 - b. Tic-Tac-Toe
 - c. Ludo
 - d. Football Penalty Shootout
 - e. Geometry
 - f. Scientific Calculator
 - g. *Dictator* the Movie
 - h. The Plot of *Mr. Robot* the Series
 - i. Pokémon Types and Battle Systems

Contest Rules

1. The outputs must match exactly as shown in the question.
2. The code should work for every sample input/output combination.
3. During the presentation, the instructor may ask any member of any team to present any of the contest problem solutions and 10 marks will be allocated for it.
4. One team should not aid any other team. If a team is struggling with something, that team can request help from the instructor.
5. One person from each team will submit the **.ipynb** file in the Google form.

Scoring Criteria

Solved Problems	10*8 = 80
Efficient and Clean Code	10
Presentation	10
Total	100

Perks and Benefits

Winning Team	+2 Points
2 nd , 3 rd and 4 th Team	+1.5 Points
5 th – 8 th Team	-1 Points
Top 8 individual performers will receive exclusive rewards	



Submission

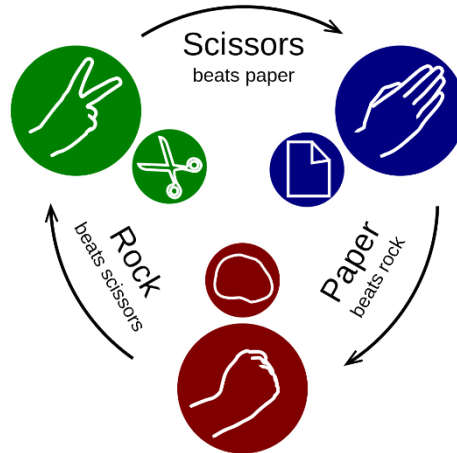
Submission Form: <https://forms.gle/W8ShQf85SPAiC55A7> (Submit using G-suite Email)

Deadline: 11:59 PM 26 June, 2022

Presentation On: 27 June, 2022 During Class

App #1: Rock-Paper-Scissor [Difficulty Level: Walk in the Park]

Imagine you are building a virtual rock-paper-scissor game playable among 3 players. Now write a python program that will take "rock", "paper" or "scissor" as inputs and decide the winner using the following conditions.



A winner is decided if:

- a) One player picks rock and the other two pick scissor
- Or b) One player picks scissor and the other two pick paper
- Or c) One player picks paper and the other two picks rock.

Remember that there **can only be one winner**. Until a winner is decided, the game will continue.

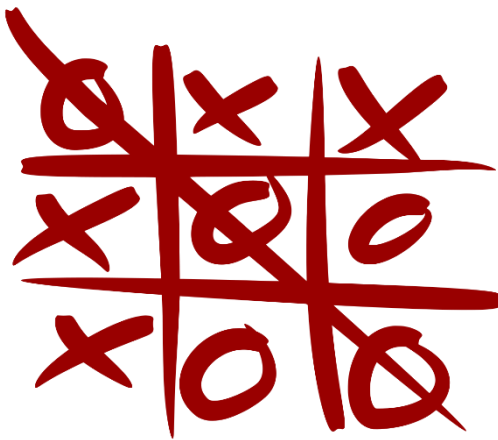
Sample Input #1: Player 1: rock Player 2: paper Player 3: rock Output: Player 2 wins Explanation: Here condition (c) was satisfied and hence player 2 became the winner.	Sample Input #2: Player 1: rock Player 2: paper Player 3: paper Player 1: scissor Player 2: scissor Player 3: paper Player 1: paper Player 2: paper Player 3: paper Player 1: paper Player 2: paper Player 3: scissor Output: Player 3 wins Explanation: Here the game continued until having paper, paper and scissor inputs, the condition (b) was satisfied and player 3 became the winner.
---	--

App #2: Tic-Tac-Toe [Difficulty Level: Walk in the Park]

Suppose, one day you ask your parents for a new phone/laptop and like traditional brown parents they start roasting you. Now, in order to protect your dignity, you have to show them what you learned studying CSE. So, you quickly decide to design a tic-tac-toe game to reestablish your dignity and ask for a MacBook/iPhone.

The rules of the game are:

- The game is played on a grid that's 3 squares by 3 squares.
- The first player to get 3 of their marks in a row (up, down, across, or diagonally) is the winner.
- When all 9 squares are full but no winner is decided, the game ends in a tie.
- The valid input range is between 1 to 9. The same input cannot be given more than once.



Sample Input #1 Player 1: 1 Player 2: 5 Player 1: 7 Player 2: 1 Cannot enter a number that is already taken. Enter again. Player 2: 6 Player 1: 4 Output: Player 1 wins!	Sample Input #2 Player 1: 2 Player 2: 5 Player 1: 0 Invalid Input. Enter again. Player 1: 3 Player 2: 1 Player 1: 7 Player 2: 9 Output: Player 2 wins!	Sample Input #3 Player 1: 2 Player 2: 7 Player 1: 3 Player 2: 1 Player 1: 4 Player 2: 5 Player 1: 9 Player 2: 6 Player 1: 8 Output: It's a tie!
---	--	--

I know, even after seeing that your program works, your parents still aren't convinced...

Sad Unemployment Noises

App #3: Aladeen Date Validator [Difficulty Level: Aladeen]

Imagine you are the chief scheduling officer of Admiral General Aladeen and are in charge of handling all his schedules.

You have to build a date validator which has to be accurate and impeccable, otherwise you will be executed and lose your head.



Take one date from the user and find out whether that date is valid (Aladeen) or not.

- You have to consider leaps years too.
- The date format (Day/Month/Year) must be followed strictly. For example, 08/02/2022.
- Invalid inputs must be handled properly. For example, 31/02/2018, 12/13/2012, -12/5/2014 and 5/0/2000 etc. are invalid dates.

Sample Input #1: Date: 12/05/1996 Output: Aladeen Date	Sample Input #2: Date: 12/13/1996 Output: Invalid Date	Sample Input #3: Date: -12/12/1996 Output: Invalid Date
Sample Input #4: Date: 29/02/2003 Output: Invalid Date	Sample Input #5: Date: 29/02/2004 Output: Aladeen Date	Sample Input #6: Date: 02/02/2004 Output: Aladeen Date

App #4: Password Validator [Difficulty Level: Walk in the Park]

Imagine you developed a split personality called Mr. Robot who is an occasional hacker. To infiltrate a large company and gain their trust, you have to create a simple password validator program for them that checks if it a password is valid or not.

The password must:

- Be of at least length 8.
- Contain at least one digit (0-9)
- Contain at least **one** upper case letter (A-Z)
- Contain at least **two** smaller case letters (a-z).

If one of the criteria is missing,
specify which one is missing.



Sample Input #1 Password: BabaSifatErTobarok21 Output: Password Valid!	Sample Input #2 Password: OmShantiOm Output: Password Invalid! Digits missing.	Sample Input #3 Password: sigmamale Output: Password Invalid! Digits missing. Upper-case missing.
Sample Input #4 Password: GGWPEZ Output: Password Invalid! Insufficient length. Digits missing. Insufficient lower-case.	Sample Input #5 Password: (/၎၎၎) (၎-')၎ (၎၎၎) _(ツ)_/ (=^_<^=) (✿◡◡) Output: Password Invalid! Digits missing. Upper-case missing. Insufficient lower-case.	Sample Input #6 Password: Dipjol#2169621 Output: Password Valid!



App #5: Triangle/Quadrilateral Validator using Sides

[Difficulty Level: Walk in the Park]

Imagine you are trying to draw a quadrilateral or a triangle. So, you are taking the lengths of each of the sides of the shape. Now, what shape you need to draw depends on the first input (3/4). After deciding what your shape will be, your job is to verify whether the shape can be drawn using the lengths given by the user.

- If no of sides: 3, a triangle is needed to be drawn. To draw a triangle, the sum of the lengths of any two sides must be greater than the third.
- If no of sides: 4, a quadrilateral is needed to be drawn. To draw a quadrilateral, the sum of the lengths of any three sides must be greater than the fourth.
- For any other input for No of angles, print “Invalid input”

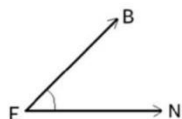
Sample Input 1: No of sides: 1 Output: Invalid input	Sample Input 2: No of sides: -2 Output: Invalid input	Sample Input 3: No of sides: 3 Length of sides: 2 3 1 Output: Invalid input
Sample Input 4: No of sides: 3 Length of sides: 2 3 2 Output: A triangle can be drawn	Sample Input 5: No of sides: 4 Length of sides: 2 3 1 4 Output: A quadrilateral can be drawn	Sample Input 6: No of sides: 4 Length of sides: 2 3 1 7 Output: Invalid input



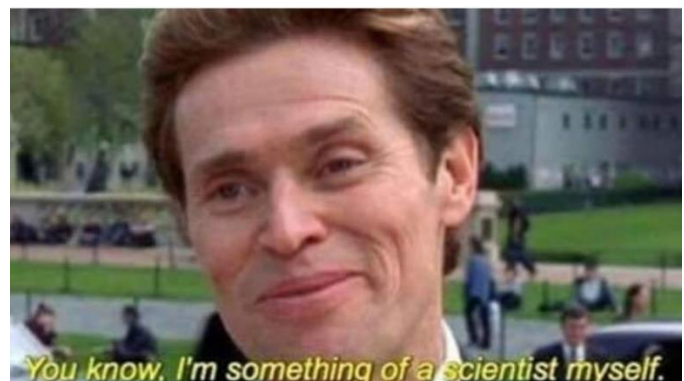
Geometry Test



Find angle BEN



When the geometry teacher starts talking about cubes and you remember how you once solved one color of a Rubik's cube



App #6: Triangle/ Quadrilateral Detector using Angles

[Difficulty Level: Walk in the Park]

Imagine you are trying to draw a quadrilateral or a triangle. So, you are taking the angles of the shape. Now, what shape you need to draw depends on the first input (3 or 4). After deciding what your shape will be, your job is to verify whether the shape can be drawn using the angles given by the user. You also have to specify what kind of triangle (equilateral/isosceles/scalene and acute/right/obtuse) or quadrilateral (rectangle/parallelogram/square/rhombus/regular quadrilateral) the shape is.

- a. If no of angles: 3, a triangle is needed to be drawn. To draw a triangle, the sum of the angles must be equal to 180 degrees.
- b. In the triangle if all the angles are equal it is an equilateral triangle, if two of the angles are equal it is an isosceles triangle, and if none of the angles are equal, it is a scalene triangle. If one of the angles is equal to 90 degrees, it is a right triangle. If one of the angles is greater than 90 degrees, it is an obtuse triangle. If none of the angles are greater than 90 degrees, it is an acute triangle.
- c. If no of angles: 4, a quadrilateral is needed to be drawn. To draw a quadrilateral, the sum of the angles must be equal to 360 degrees.
- d. In the quadrilateral, if all the angles are equal (90 degrees) it is either a square or a parallelogram, if the first-third and second-fourth angles are equal, it is a rhombus or a rectangle. Otherwise, it is a regular quadrilateral.
- e. Angles cannot be zero or negative.
- f. For any other input for No of angles, print “Invalid input”.

Sample Input #1: No of angles: 1 Output: Invalid input	Sample Input 2: No of sides: 4 Angles: 80 100 80 100 Output: A rhombus or a rectangle can be drawn	Sample Input 3: No of angles: 3 Angles: 60 90 50 Output: Invalid input
Sample Input 4: No of sides: 3 Angles: 60 90 30 Output: A scalene right triangle can be drawn	Sample Input 5: No of sides: 4 Angles: 60 80 70 150 Output: A regular quadrilateral can be drawn	Sample Input 6: No of sides: 4 Angles: 60 80 70 130 Output: Invalid input

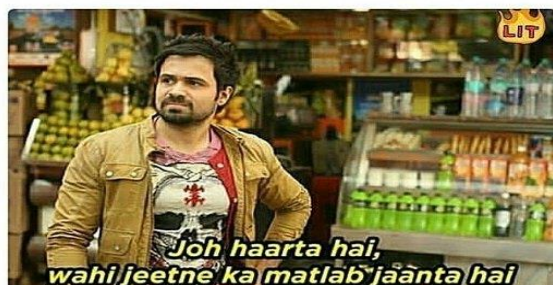
App #7: Ludo [Difficulty Level: Nose to the Grindstone]

Imagine you are designing a minimalistic version of the game ludo with the following conditions:

- a. The player who crosses throughout the board of exactly 25 boxes wins.
- b. Each of the players get to roll the dice and can get from 1-6. If a player scores 6, he/she gets another immediate turn. If a player scores 6 three times in a row, all three turns get cancelled and the player stays stationary.
- c. If a player crosses 24 boxes, he/she has to get exactly 1 after rolling the dice in order to win. Otherwise, he/she will stay in the same place.
- d. The one who gets to the 25th box first will win.
- e. Until a winner is decided, the dice will be rolled between the players and the game will continue.
- f. If one player gets to a box which is already occupied by another player, the one who occupied the box gets eaten and has to start from box no 1 again.

Sample Input #1: 1. Player 1: 5 2. Player 2: 4 3. Player 1: 5 4. Player 2: 5 5. Player 1: 6 6. Player 1: 1 7. Player 2: 4 8. Player 1: 4 9. Player 2: 6 10. Player 2: 6 Output: Player 2 wins	Sample Input #2: 1. Player 1: 5 2. Player 2: 3 3. Player 1: 5 4. Player 2: 2 5. Player 1: 5 6. Player 2: 4 7. Player 1: 5 8. Player 2: 5 9. Player 1: 3 10. Player 2: 4 11. Player 1: 5 12. Player 2: 4 13. Player 1: 1 14. Player 2: 6 15. Player 1: 3 16. Player 2: 4 17. Player 1: 1 Output: Player 1 wins	Sample Input #3: 1. Player 1: 2 2. Player 2: 3 3. Player 1: 6 4. Player 1: 6 5. Player 1: 6 6. Player 2: 3 7. Player 1: 5 8. Player 2: 3 9. Player 1: 5 10. Player 2: 6 11. Player 2: 6 12. Player 2: 4 Output: Player 2 wins [Hint: Player 1 hits 3 6s in a row and therefore his turn gets cancelled and he stays stationary]	Sample Input #4: 1. Player 1: 4 2. Player 2: 5 3. Player 1: 4 4. Player 2: 5 5. Player 1: 4 6. Player 2: 5 7. Player 1: 4 8. Player 2: 5 9. Player 1: 4 10. Player 2: 5 11. Player 1: 4 12. Player 2: 5 13. Player 1: 1 Output: Player 1 wins [Hint: Player 2 gets eaten in turn no 9 and starts from box 1 again]
---	--	--	--

How I console myself every time I lose in ludo.



Me after getting 6 6 5 in ludo star.



App #8: Super Calculator [Difficulty Level: Nose to the Grindstone]

Design a super calculator that takes two/one numbers as inputs what operation is going to be performed in between these numbers. Whether you shall be taking two numbers or only one number as input will depend on the operation you need to perform. Your super calculator should be able to perform:

- a. Addition
- b. Subtraction
- c. Multiplication
- d. Division
- e. Floor division
- f. Exponentiation/Root Operation
- g. Modulus
- h. Negation (Sign conversion, Single input)
- i. Compare (Whether first number is equal to, greater or less than the second number)
- j. State of the number (Whether the number is positive, negative or zero. Single input)

Your program should be able to handle invalid inputs such as:

- a. Division by 0
- b. Root of negative numbers
- c. Any input other than digits
- d. The first input not being within 0-9



Sample Input #1 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 0 First Number: 5 Second Number: -2 Output: Addition: 3	Sample Input #10 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 5 First Number: -25 Second Number: 0.5 Output: Invalid Input: Root of negative number
Sample Input #2 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 1 First Number: 5 Second Number: -2 Output: Subtraction: 7	Sample Input #11 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 6 First Number: 25 Second Number: 0 Output: Invalid Input: Division by zero
Sample Input #3 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 2 First Number: 5 Second Number: -2 Output: Multiplication: -10.0	Sample Input #12 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 6 First Number: 25 Second Number: 6 Output: Modulus: 1

Sample Input #4 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 3 First Number: 5 Second Number: 0 Output: Invalid Input: Division by zero	Sample Input #13 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 7 Number: 15 Output: Negation: -15
Sample Input #5 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 3 First Number: 5 Second Number: -2 Output: Division: -2.50	Sample Input #14 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 7 Number: -15 Output: Negation: 15
Sample Input #6 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 4 First Number: 5 Second Number: 2 Output: Floor division: 2	Sample Input #15 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 8 First Number: 25 Second Number: 12 Output: First Number is greater than second number
Sample Input #7 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 4 First Number: 5 Second Number: 0 Output: Invalid Input: Division by zero	Sample Input #16 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 8 First Number: 25 Second Number: 25 Output: Both numbers are equal
Sample Input #8 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 5 First Number: 5 Second Number: 2 Output: Exponentiation/Root: 25.0	Sample Input #17 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 9 Number: 25 Output: The number is positive
Sample Input #9 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 5 First Number: 25 Second Number: 0.5 Output: Exponentiation/Root: 5.0	Sample Input #18 Which Operation you want to perform? (0=Addition, 1=Subtraction, 2=Multiplication, 3=Division, 4= Floor division, 5= Exponentiation/ Root Operation, 6= Modulus, 7= Negation, 8= Compare, 9= State of the number): 10 Output: Invalid operation choice

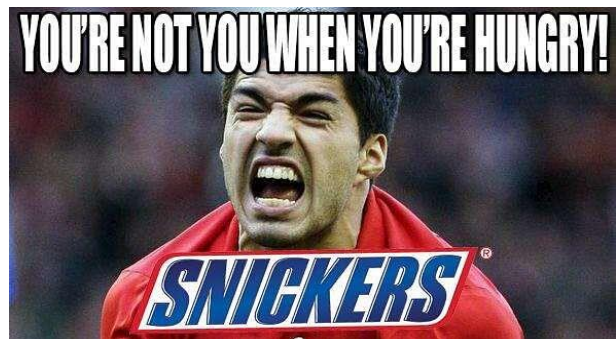
App #9: Penalty Shootout [Difficulty Level: Nose to the Grindstone]

Imagine two teams are playing football and the scores are tied after 120 minutes of play. Now the winners will be decided through a penalty shootout. Now write a python program that takes in "goal" or "miss" as inputs and prints the winner and the score using the following conditions.

1. Each team will get at most 5 chances to shoot. The team scoring more goals wins.
2. If the goals are tied again, a draw is declared.
3. A Team may be able to secure victory even before all 5 chances are used. For example, if team 1 scores the first 3 goals and team 2 misses the first 3 goals, the game ends instantly and Team 1 wins.

Sample Input #1: Team 1: goal Team 2: goal Team 1: goal Team 2: goal Team 1: miss Team 2: goal Team 1: goal Team 2: miss Team 1: goal Team 2: miss Output: Team 1 wins by 4-3 Explanation: After 5 turns, team 1 scored 4 goals and team 2 scored 3. Therefore, team 1 are the winners.	Sample Input #2: Team 1: goal Team 2: miss Team 1: goal Team 2: goal Team 1: miss Team 2: goal Team 1: miss Team 2: goal Team 1: goal Team 2: miss Output: Draw by 3-3 Explanation: After 5 turns, both teams scored an equal number of goals (3 goals each). Therefore, a draw is declared.	Sample Input #3: Team 1: miss Team 2: goal Team 1: goal Team 2: goal Team 1: miss Team 2: goal Team 1: miss Output: Team 2 wins by 3-1 Explanation: Since team 1 only had one chance left but team 2 is already leading by 3-1. Therefore, no further input is needed and Team 2 is declared the winner.
---	--	---

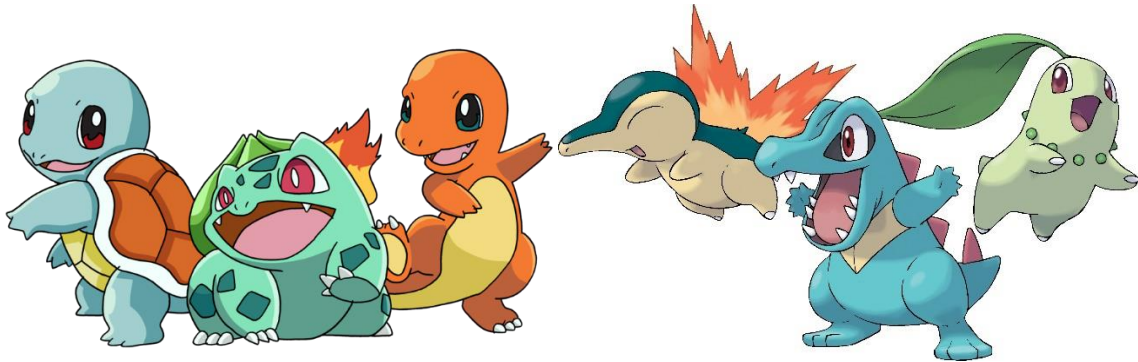
Someone: *breathes*
Neymar:



App #10: Pokémon Battle [Difficulty Level: Nose to the Grindstone]

Imagine on one morning you woke up in the world of Pokémon as Ash Ketchum. Professor Oak then asked you to choose one from the following 6 starter Pokémon. After you chose your Pokémon, your arch nemesis Gary Oak chose one from the remaining Pokémon.

Now Gary being the bully that he is, challenges you to a battle. However, you are a CSE student in real life and therefore, decided to design a Pokémon battle simulator to determine your winning chances.



- Charmander and Cyndaquil are fire type, Squirtle and Totodile are water type, and Bulbasaur and Chikorita are grass type.
- Fire types are strong against grass type and do double damage, water types are strong against fire type and do double damage, and grass types are strong against water types and do double damage. In these cases, type affinity = 2
- Fire types are weak against water type and do half damage, water types are weak against grass types and do half damage, grass types are weak against fire types and do half damage. In these cases, type affinity = 0.5
- For same types (fire-fire/water-water/grass-grass), type affinity = 1
- Assume all 6 Pokémon have 50 hp and 20 attack power.
- The battle goes on until one of the Pokémon's hp goes down to zero (or negative). Equation used to calculate remaining hp after each turn:

$$\text{Remaining hp} = \text{previous hp} - \text{attack power} * \text{type affinity}$$

- When the battle is over, declare who is the winner.

Your task is to create a Pokémon battle simulator that determines who wins between you and your rival.

Sample Input #1 You Choose: Charmander Gary Chooses: Totodile Output: === Turn 1 === Charmander has 10 hp left Totodile has 40 hp left === Turn 2 === Charmander has 0 hp left Totodile has 30 hp left Totodile and Gary wins!	Sample Input #2 You Choose: Squirtle Gary Chooses: Cyndaquil Output: === Turn 1 === Squirtle has 40 hp left Cyndaquil has 10 hp left === Turn 2 === Squirtle has 30 hp left Cyndaquil has 0 hp left Squirtle and Ash wins!	Sample Input #3 You Choose: Chikorita Gary Chooses: Bulbasaur Output: === Turn 1 === Chikorita has 30 hp left Bulbasaur has 30 hp left === Turn 2 === Chikorita has 10 hp left Bulbasaur has 10 hp left === Turn 3 === Chikorita has 0 hp left Bulbasaur has 0 hp left It's a tie!
--	--	---